

601.433/633 Intro Algorithms

Recitation 1

Fall 2026

Asymptotic Analysis

For each of the following statements explain if it true or false and prove your answer. The base of log is 2 unless otherwise specified, and $\ln$ is $\log_e$.

1. $2^{5 \log n} = O(n^2)$

False. For the sake of contradiction, assume that the statement is true. Then there exist constants $c, n_0 > 0$ such that for all $n > n_0$, $2^{5 \log n} \leq c \cdot n^2$. Observe that $2^{5 \log n} = (2^{\log n})^5 = n^5$. We will consider two cases: $c \geq 1$, and $0 < c < 1$. For any constant $c \geq 1$, and for any $n > c$, we have that $n^5 > n^3 > c \cdot n^2$. Choosing $n > \max\{c, n_0\}$ contradicts our assumption that there exists an n_0 such that for all $n > n_0$, $2^{5 \log n} \leq c \cdot n^2$. And for any $0 < c < 1$, and any $n > 1$, we have that $n^5 > n^2 > c \cdot n^2$. Choosing $n > \max\{1, n_0\}$ again gives us a contradiction to our assumption. So for all values of $c > 0$, it is impossible to find a value of $n_0 > 0$ that satisfies the definition of big O .

2. $\sqrt{n + \cos n} = O(\sqrt{n})$

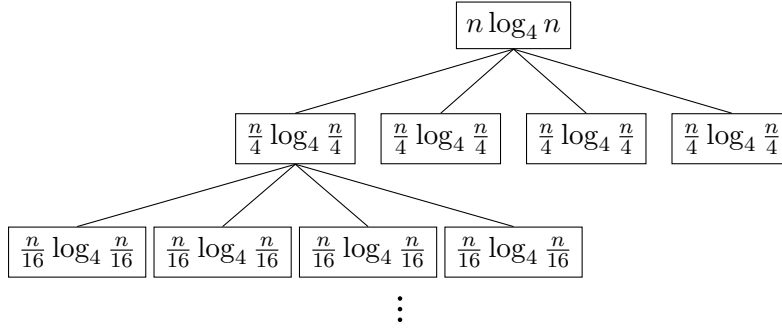
True. We need to show that there exist constants $c, n_0 > 0$ such that $\sqrt{n + \cos n} \leq c\sqrt{n}$ for all $n > n_0$. We know that $\cos n \leq 1$ for all values of n , and hence $\sqrt{n + \cos n} \leq \sqrt{n + 1}$ for all $n > 1$. If we set $n_0 = 1$, then for all $n > n_0$ we have that $\sqrt{n + \cos n} \leq \sqrt{n + 1} \leq \sqrt{n + 3n} = 2\sqrt{n}$. Thus, if we choose $c = 2$ and $n_0 = 1$, these functions satisfy the definition of big O .

Recurrences

Solve the following recurrences, giving your answer in Θ notation (so prove both an upper bound and a lower bound). For each of them you may assume $T(x) = 1$ for $x \leq 5$ (or if it makes the base case easier you may assume $T(x)$ is any other constant for x at most some constant). Show your work.

$$T(n) = 4T(n/4) + n \log_4 n$$

Solution: We can solve this by drawing out the recursion tree:



To make our analysis easier, we will assume $T(x) = 1$ for $x \leq 4$. Suppose n is an exact power of four. Then, as you can see, the tree will have exactly $h = (\log_4 n - 1)$ levels (since we want $1 < \frac{n}{4^h} \leq 4$) *below the root*. If we don't assume that n is a power of four, then the height of the tree is $h = \lceil \log_4 n \rceil - 1$ *not including the root*. It is a good exercise to work this out with concrete examples. At the i -th level ($0 \leq i < h$), there will be 4^i nodes, each with value $\frac{n}{4^i} \log_4 \frac{n}{4^i}$. Thus, the total work at level i is $n \log_4 \frac{n}{4^i}$. In the bottom layer at level h , there will be 4^h leaves, each with value 1 (since it's the base case). Note that since h is the unique integer in $[\log_4 n - 1, \log_4 n)$, the total number of leaves 4^h is the unique power of 4 in $[\frac{n}{4}, n)$, which is also the total work at level h . We sum the work from level 0 to h , to get an upper bound on $T(n)$:

$$\begin{aligned}
 T(n) &= 4^h + \sum_{i=0}^{h-1} n \log_4 \frac{n}{4^i} \\
 &< n + \sum_{i=0}^{h-1} n \log_4 \frac{n}{4^i} = n + n \sum_{i=0}^{h-1} \log_4 \frac{n}{4^i} \\
 &= n + n \sum_{i=0}^{h-1} (\log_4 n - \log_4 4^i) \\
 &= n + n \sum_{i=0}^{h-1} (\log_4 n - i) \\
 &\leq n + n \sum_{i=0}^{h-1} (\log_4 n) \\
 &= n + nh(\log_4 n) < n + n \log_4^2 n
 \end{aligned}$$

Using the fact $h < \log_4 n$.

Hence, $\boxed{T(n) = O(n \log^2 n)}$.

On the other hand, to lower bound $T(n)$, one can consider all levels except the leaves: that is, we sum the work from $i = 0$ to $i = h - 1$, ignoring the work in the base case level. Thus,

$$\begin{aligned}
T(n) &\geq \sum_{i=0}^{h-1} n \log_4 \frac{n}{4^i} \\
&= n \sum_{i=0}^{h-1} (\log_4 n - i) \\
&= n \left(h \log_4 n - \sum_{i=0}^{h-1} i \right) \\
&= n \left(h \log_4 n - \frac{h(h-1)}{2} \right)
\end{aligned}$$

Using the identity $\sum_{i=1}^n i = \frac{n(n+1)}{2}$

the above term can be viewed as a function of h . Verify that when $h < \log_4 n$, the function is increasing.

$$\begin{aligned}
&\geq n(\log_4 n - 1) \left(\log_4 n - \frac{\log_4 n - 2}{2} \right) && \text{Using the fact } h \geq \log_4 n - 1 \\
&= n(\log_4 n - 1) \left(\frac{\log_4 n}{2} + 1 \right)
\end{aligned}$$

This gives us that $\boxed{T(n) = \Omega(n \log^2 n)}$.

Therefore, we can conclude that

$$\boxed{T(n) = \Theta(n \log^2 n)}.$$

Selection from Two Sorted Arrays.

Let A and B be two sorted arrays, each containing n comparable elements. Assume that $1 \leq k \leq n$. Design an algorithm that finds the k -th smallest element in the combined multiset $A \cup B$ without explicitly merging the two arrays. In particular, find a function $f(n) = o(n)$ and design a deterministic algorithm whose running time (measured in terms of the number of comparisons) is $O(f(n))$. Prove the running time and correctness of your algorithm.

Hint: Compare elements near rank $\lfloor \frac{k}{2} \rfloor$ in the two arrays and determine which portions of the arrays can be safely discarded.

Solution:

The following algorithm finds the k -th smallest element of $A \cup B$ in $O(\log n)$ time (here we assume that the arrays are indexed starting at 1):

```
KTHELEMENT( $A, B, k$ ) :  
    if  $k = 1$  then  
        return  $\min\{A[1], B[1]\}$   
     $t \leftarrow \lfloor k/2 \rfloor$   
     $s \leftarrow k - t$   
    if  $A[t] \leq B[t]$  then  
        return KTHELEMENT( $A[t+1 : |A|], B[1 : s], s$ )  
    else  
        return KTHELEMENT( $A[1 : s], B[t+1 : |B|], s$ ).
```

Correctness.

We can prove that this algorithm is correct by induction on k . Our induction hypothesis is as follows: Assume that for every $k < \ell$, and for any two sorted arrays each containing at least k elements, the algorithm correctly returns the k -th smallest element of their union. In the base case, where $k = 1$ and both arrays have at least one element, the algorithm works correctly.

Consider the inductive case, where $k = \ell$. Suppose that at the start of a call KTHELEMENT(A, B, k) where $k = \ell$, both arrays are sorted and contain at least k elements. This precondition is true initially because both arrays are sorted and have length n , where $k \leq n$. Set $t = \lfloor \frac{\ell}{2} \rfloor$, and $s = \ell - t = \lceil \frac{\ell}{2} \rceil$. Suppose without loss of generality that $A[t] \leq B[t]$. We first show that the elements $A[1], A[2], \dots, A[t]$ may be discarded. Let x be the ℓ -th smallest value of $A \cup B$. Since $A[t] \leq B[t]$, at most $t - 1$ elements of A and at most $t - 1$ elements of B are strictly smaller than $A[t]$. Hence at most $2t - 2 < \ell$ elements are strictly smaller than $A[t]$. Therefore $A[t]$ is no larger than the ℓ -th order statistic x . If $A[t] < x$, then every element of $A[1 : t]$ is strictly smaller than x , so we can safely remove them. x now becomes the $\ell - t = s$ -th smallest element after the deletion. If $A[t] = x$, then after deleting $A[1 : t]$, at least $\ell - t = s$ elements remain that are no larger than x . Moreover, all remaining elements strictly smaller than x must lie in $B[1 : t - 1]$, so there are at most $t - 1 < s$ such elements. Hence x is still the s -th smallest element after the deletion.

We can also discard every element of B after $B[s]$. Consider the first such element, $B[s + 1]$. Since

$s \geq t$, $B[t] \leq B[s+1]$. Because $A[t] \leq B[t]$, all of $A[1], \dots, A[t]$ are no larger than $B[s+1]$. In addition, $B[1], \dots, B[s]$ are no larger than $B[s+1]$. Thus there are already $t + s = \ell$ elements no larger than $B[s+1]$. Hence $B[s+1]$, and every element after it, is no smaller than the ℓ -th smallest element x . Thus we can remove $B[s+1], B[s+2], \dots, B[|B|]$.

Therefore, we now need to find the s -th smallest element of $A[t+1 : |A|] \cup B[1 : s]$. The recursive call is on $A' = A[t+1 : |A|]$ and $B' = B[1 : s]$. We need to show that the precondition is satisfied for both the arrays for the inductive hypothesis to hold: that is, we need to show that both A' and B' have at least s elements. For B' , this is immediate. For A' , we remove exactly t elements from A . Since the current call satisfies $|A| \geq \ell$, we have $|A'| = |A| - t \geq \ell - t = s$. The case $A[t] > B[t]$ is symmetric. Hence, from the inductive assumption, we can conclude that the algorithm correctly returns the ℓ -th smallest element of $A \cup B$.

Running Time.

Each call to KTHELEMENT does a constant amount of work before recursing. Further, at every recursive step, the new desired order statistic is

$$s = k - \left\lfloor \frac{k}{2} \right\rfloor = \left\lceil \frac{k}{2} \right\rceil.$$

Hence the recurrence is

$$T(k) = T\left(\left\lceil \frac{k}{2} \right\rceil\right) + O(1).$$

Therefore,

$$T(k) = O(\log k).$$

Since $k \leq n$,

$$\boxed{T(k) = O(\log n)}.$$